

Assignment 1. Cryptography

Project Release: March 24 Tue, 2026 KST

Deadline: April 6 Mon, 2026 by 8:59:59pm KST

The code and other answers you submit must be entirely your own work. You may discuss this assignment in small groups with classmates, but you must write your code and solutions yourself. Please acknowledge any collaborators you discussed the assignment with and cite any references you used. You may consult published references, provided that you cite them appropriately (e.g., in program comments).

Solutions must be submitted via Blackboard.

Introduction

In this project, we will begin by investigating many-time pad attacks, then move on to length-extension attacks in widely used cryptographic hash functions and an implementation vulnerability in a popular digital signature scheme.

Start early! This assignment will take longer than you think.

Part 1: Many-time pad attack (5pts)

A many-time pad attack becomes possible when the same one-time pad key is reused to encrypt multiple plaintexts. A one-time pad is perfectly secure only if the key is truly random, at least as long as the message, and never reused. If the same key is used more than once, this security guarantee is lost. In particular, if two ciphertexts c_1 and c_2 are produced from plaintexts m_1 and m_2 using the same key k , then

$$c_1 = m_1 \oplus k \quad \text{and} \quad c_2 = m_2 \oplus k.$$

XORing the two ciphertexts cancels out the key and gives

$$c_1 \oplus c_2 = m_1 \oplus m_2.$$

Although this does not directly reveal either plaintext, it leaks valuable information about their relationship. When the plaintexts are English text, an attacker can exploit properties such as **spaces, common words, and predictable character frequencies** to infer portions of the plaintexts and recover key bytes. Once some key bytes are recovered, they can be used to decrypt the corresponding positions in every ciphertext encrypted under the same pad.

You are given the ciphertext file `ctxt.txt` on Blackboard. Each line of the ASCII plaintext was encrypted by XORing it with the same randomly generated one-time pad key and then encoded in hex. Your task is to

decrypt as much of the plaintext as possible. At a minimum, you should be able to recover the entire last line.

Hint. All plaintexts consist of lowercase letters, and their contents are related to security.

What to submit. A Python 3.x script named `many_time_pad.py` that: Reads a file `ctxt.txt`. Outputs the corresponding plaintext. It does not need to be fully automated, since that is quite challenging. This exercise will be graded manually, so your code should be readable. Poorly written code may cause misunderstandings during grading, and any resulting penalty will be your responsibility. Second, `wirteup.pdf` that explains your workflow step by step. You are given a Latex template on Blackboard.

Additional material. For reference, the ciphertext was generated using the following Python 3 program, adapted from Dan Boneh.

```
import os

MSGs = open("hw1-message.txt").readlines()

def byt xor(a, b):      # xor two byte arrays (trims the longer input)
    return bytes([x ^ y for (x,y) in zip (a,b)])

def random(size=16):
    return os.urandom(size)

def encrypt(key, msg):
    c = byt xor(key, msg)
    print(c.hex())
    return c

def main():
    key = random(1024)
    ciphertexts = [encrypt(key, msg.encode('utf-8')) for msg in MSGs]
    return ciphertexts

if __name__=='__main__':
    main()
```

Part 2: Hash length extension (4pts)

In most applications, you should use MACs such as HMAC-SHA256 instead of plain cryptographic hash functions (e.g., MD5, SHA-1, or SHA-256) because hashes, also known as digests, fail to match our intuitive security expectations. What we really want is something that behaves like a pseudorandom function, which HMACs seem to approximate and hash functions do not.

One difference between hash functions and pseudorandom functions is that many hashes are subject to length extension. Many common hash functions use a design called the Merkle-Damgard construction. Each is built around a compression function f and maintains an internal state s , which is initialized to a fixed constant. Messages are processed in fixed-size blocks by applying the compression function to the current state and current block to compute an updated internal state, i.e., $s_{i+1} = f(s_i, b_i)$. The result of the final application of the compression function becomes the output of the hash function.

A consequence of this design is that if we know the hash of an n -block message, we can find the hash of

longer messages by applying the compression function for each block b_{n+1} , b_{n+2} , ... that we want to add. This process is called length extension, and it can be used to attack many applications of hash functions.

2a. Experimenting

To experiment with this idea, we'll use a Python implementation of the MD5 hash function, though SHA-1 and SHA-256 are vulnerable to length extension in the same way. You can download the `pymd5` module [here](#) and learn how to use it by running `pydoc pymd5`. To follow along with these examples, run Python in interactive mode and run the command `from pymd5 import md5, padding`.

Consider the string "Use HMAC, not hashes". We can compute its MD5 hash by running:

```
from pymd5 import md5, padding
m = "Use HMAC, not hashes"
h = md5()
h.update(m)
print(h.hexdigest())
or, more compactly,

print(md5(m).hexdigest())
```

The output should be `3ecc68efa1871751ea9b0b1a5b25004d`.

MD5 processes messages in 512-bit blocks, so internally, the hash function pads `m` to a multiple of that length. The padding consists of the bit 1, followed by as many 0 bits as necessary, followed by a 64-bit count of the number of bits in the unpadded message. (If the 1 and the count won't fit in the current block, an additional block is added.) You can use the function `padding(count)` in the `pymd5` module to compute the padding that will be added to a count-bit message.

Even if we didn't know `m`, we could compute the hash of longer messages of the general form `m + padding(len(m)*8) + suffix` by setting the initial internal state of our MD5 function to `MD5(m)`, instead of the default initialization value, and setting the function's message length counter to the size of `m` plus the padding (a multiple of the block size). To find the padded message length, guess the length of `m` and run `bits = (length_of_m + len(padding(length_of_m * 8))) * 8`.

The `pymd5` module lets you specify these parameters as additional arguments to the `md5` object:

`h = md5(state=bytes.fromhex("3ecc68efa1871751ea9b0b1a5b25004d"), count=512)` Now you can use length extension to find the hash of a longer string that appends the suffix "Good advice". Simply run:

```
x = "Good advice"
h.update(x)
print(h.hexdigest())
```

to execute the compression function over `x` and output the resulting hash. Verify that it equals the MD5 hash of `m + padding(len(m)*8) + x`. Notice that, due to the length-extension property of MD5, we didn't need to know the value of `m` to compute the hash of the longer string - all we needed to know was `m`'s length and its MD5 hash.

This part of the assignment is intended to introduce length extension and familiarize you with the Python MD5 module we will be using; you will not need to submit anything for it.

2b. Conduct a length extension attack

Length extension attacks can cause serious vulnerabilities when people mistakenly try to construct something like an HMAC by using `hash(secret || message)`; ‘||’ means concatenation. The Bank of CSE 467, which is not up-to-date on its security practices, hosts an API that allows its client-side applications to perform actions on behalf of a user by loading URLs of the form:

```
https://cse467.com/pa1/api?token=2759e37e9071ecc7048a75c77dfbffa2&user=minkyung&command1=ListSquirrels&command2=NoOp
```

where token is MD5(user’s 8-character password || user=... [the rest of the decoded URL starting from user= and ending with the last command]).

Using the techniques that you learned in the previous section and without guessing the password, apply length extension to create a URL ending with `&command3=UnlockAllSafes` that would be treated as valid by the server API.

Note: Because of its bad security practices, the Bank of CSE 467 has taken down its website.

Hint: You might want to use the `quote()` function from Python’s `urllib.parse` module to encode non-ASCII characters in the URL. Also consider using `bytes.fromhex()` when passing in the state to `md5()`.

Historical fact: In 2009, security researchers found that the API used by the photo-sharing site Flickr suffered from a length-extension vulnerability almost exactly like the one in this exercise.

What to submit. A Python 3.x script named `len_ext_attack.py` that:

Accepts a valid URL in the same form as the one above as a command line argument. Modifies the URL so that it will execute the `UnlockAllSafes` command as the user. Prints the new URL to the command line. You should make the following assumptions:

The input URL will have the same form as the sample above, but we may change the server hostname and the values of token, user, command1, and command2. These values may be of substantially different lengths than in the sample.

The input URL may be for a user with a different password, but the length of the password will be unchanged.

You can base your code on the following example:

```
import sys, urllib.parse
from pymd5 import md5, padding
url = sys.argv[1]
```

Your code to modify url goes here

```
print(new_url)
```

Part 3: RSA signature forgery (4pts)

A secure implementation of RSA encryption or digital signatures requires a proper padding scheme. RSA without padding, also known as textbook RSA, has several undesirable properties. For example, it is trivial

for an attacker with only an RSA public key pair (n, e) to produce a mathematically valid message, signature pair by choosing an s and returning (s^e, s) .

In order to prevent an attacker from being able to forge valid signatures in this way, RSA implementations use a padding scheme to provide structure to the messages that are encrypted or signed. The most commonly used padding scheme in practice is defined by the PKCS #1 v1.5 standard, which can be found at <https://tools.ietf.org/html/rfc2313>. The standard defines, among other things, the format of RSA keys and signatures and the procedures for generating and validating RSA signatures.

3a. Validating RSA signatures

You can experiment with validating RSA signatures yourself. Create a file called `key.pub` that contains the following RSA public key:

```
-----BEGIN PUBLIC KEY-----
MIGdMA0GCSqGSIb3DQEBAQUAA4GLADCBhwKBgQDWl2CtskRLU4M8juE0xcJ7kGFg
3I1WMs0oqfqz2h15R2DOih9K0hwZ3EzORWiV0/fWZsNNyeJabs7gvFQP0zaf0k0y
L04gS+Bsm1M4PEyVqGVLqfFACfLurzDaessC15EF00WiJ60XVAWcRVRTYo14HJqM
rhDy+r1X78UnHIj9TwIBAw==
-----END PUBLIC KEY-----
```

You can view the modulus and public exponent of this key by running:

```
openssl rsa -in key.pub -pubin -text -noout
```

Create a file containing only the text CSE 467 rul3z!.

```
echo -n 'CSE 467 rul3z!' > message
```

The following is a base64-encoded signature of the file message using the public key above.

```
bPLUtEldzAZq60lMPCzbuqNajp0Topee8A3PK+m4+aapeKVS4/CCQIExmR68cQ/hwh4MJNezJDEQcj9F5gnutGLTxcgy
YTtp8khWU+K+xxMja09hW0KhlpvkC7VGjLeSpDPoohYmizD0Ckqs4gDtc2RSEo0ak47RuobkslnZKE=
```

Convert this signature into a binary file:

```
base64 -d -i sig.b64 > sig
```

Now verify the signature against the file you created.

```
openssl dgst -sha1 -verify key.pub -signature sig message
```

We can also use basic math operations in Python to explore this signature further. Remember, RSA ciphertexts, plaintexts, exponents, moduli, and signatures are actually all integers.

Open a Python shell and run the following commands to import the signature as an integer:

```
from Crypto.PublicKey import RSA
from Crypto.Hash import SHA
signature = int(open('sig', 'rb').read().hex(), 16)
```

Next, import the public key file that you created earlier:

```
pubkey = RSA.importKey(open('key.pub').read())
```

The modulus and exponent are then accessible as `pubkey.n` and `pubkey.e`, respectively. Now reverse the signing operation and examine the resulting value in hex:

```
"{:0256x}".format(pow(signature, pubkey.e, pubkey.n))
```

You should see something like '0001fff...3071747a5774d00c55515fa4e1c8f4cda8be4a97'. Verify that the last 20 bytes of this value match the SHA-1 hash of your file:

```
SHA.new(b"CSE 467 rul3z!").hexdigest()
```

You don't need to submit anything for this part.

4b. PKCS #1 v1.5 signature padding

The signed value you examined in the previous section had been padded using the PKCS #1 v1.5 signature scheme. PKCS #1 v1.5 padding for RSA signatures is structured as follows: one 00 byte, one 01 byte, some FF bytes, another 00 byte, some special ASN.1 bytes denoting which hash algorithm was used to compute the hash digest, then the bytes of the hash digest itself. The number of FF bytes varies such that the size of `m` is equal to the size of the RSA key.

A `k`-bit RSA key used to sign a SHA-1 hash digest will generate the following padded value of `m`:

```
00 01 FF...FF 00 3021300906052B0E03021A05000414 XX...XX
      ^^^^^      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
      k/8 - 38 bytes wide      ||      20-byte SHA-1 digest
                               ASN.1 "magic" bytes
```

When PKCS padding is used, it is important for implementations to verify that every bit of the padded, signed message is exactly as it should be. It is tempting for an implementer to validate the signature by first stripping off the 00 01 bytes, then some number of padding FF bytes, then 00, and then parse the ASN.1 and verify the hash. If the implementation does not check the length of the FF bytes and that the hash is in the least significant bits of the message, then it is possible for an attacker to forge values that pass this validation check.

This possibility is particularly troubling for signatures generated with $e = 3$. If the length of the required padding, ASN.1 bytes, and hash value is significantly less than $n^{1/3}$ then an attacker can construct a cube root over the integers whose most significant bits will validate as a correct signature, ignoring the actual key. To construct a "signature" that will validate against such implementations, an attacker simply needs to construct an integer whose most significant bytes have the correct format, including the hashed message, pad the remainder of this value with zeros or other garbage that will be ignored by the vulnerable implementation, and then take a cube root over the integers, rounding as appropriate.

3c. Constructing forged signatures

The Bank of CSE 467 has a website that its employees use to initiate wire transfers between bank accounts. To authenticate each transfer request, the control panel requires a signature from a particular 2048-bit RSA key:

```
-----BEGIN PUBLIC KEY-----
MIIBIDANBgqhkiG9w0BAQEFAAOCAQ0AMIIBCAKCAQEAxWbNtYpsNkBN0TNKtezw
1KoEoU+SVa72EZgstMq1GHF9DAGn1RHV1PTdkf4uc3avIYhep5gCa70bVAcZhBEj
```

```
PZrG+kb7+lK3m0lCBiY4L5lXPVUy4ihNQfla09081RbB3QFav1KPB/0677QVLrOp
I+bY0imF6yNu0dxHCJsS0cE0hu+1jc5rAppIG05H/sAlutQGtNA0xSU0/4n8Y/RR
oxUNxcH1NGhxftdnYMdKgsA7YgzJk68lqyHAnEd1tAwYQZNaXFuzO3mcMsSmmYRl
JDIV/W7tkTRtv0FlCYb5JFGsYIbqIQ6qauk9/La8MgyqhMbbw1VjPJ4WDf0INIXE
RQIBAw==
-----END PUBLIC KEY-----
```

Unfortunately, this control panel is running old software that has not been patched to fix the signature forgery vulnerability.

Using the signature forgery technique described above, produce an RSA signature that validates against the Bank of CSE 467 site.

Historical fact: This attack was discovered by Daniel Bleichenbacher, who presented it in a lightning talk at the rump session at the Crypto 2006 conference. His talk is described in this mailing list posting: <https://mailarchive.ietf.org/arch/msg/openpgp/5rnE9ZRN1AokBVj3VqblG1P63QE/>. At the time, many important implementations of RSA signatures were discovered to be vulnerable to this attack, including OpenSSL. In 2014, the Mozilla library NSS was found to be vulnerable to this type of attack: <https://www.mozilla.org/security/advisories/mfsa2014-73/>.

What to submit. A Python 3.x script called `bleichenbacher.py` that:

Accepts a double-quoted string as command-line argument. Prints a base64-encoded forged signature of the input string. We have provided a Python library, `roots.py`, that provides several useful functions that you may wish to use when implementing your solution. You can download the library [here](#). Your program may assume that `PyCrypto` and `roots.py` are available, and may use standard Python libraries, but should otherwise be self-contained.

In order to use these functions, you will have to import `roots.py`. You may wish to use the following template:

```
from Crypto.PublicKey import RSA
from Crypto.Hash import SHA
from roots import *
import sys
```

```
message = sys.argv[1]
```

Your code to forge a signature goes here.

```
some example functions from roots
root, is_exact = integer_nthroot(27, 3)
print(integer_to_base64(root).decode())
```

Submission checklist

- `many_time_pad.py` (part1)
- `len_ext_attack.py` (part2)
- `bleichenbacher.py` (part3)
- `writeup.pdf` (part1). A LaTeX-formatted write-up as a PDF explaining your workflow step by step. Keep it concise: the max length is 2 pages including figures, or 1 pages if no figures are included. It is required to use the Latex template.